- Blueprint Grid System - Setup Instructions
  - Overview
  - Step 1: Scene Setup
    - A. Create Grid Manager
    - B. Create Grid Visuals
    - C. Create Interaction Handler
  - Step 2: Material Setup
    - Recommended Shader: Unlit/Transparent
    - Shader Tips:
  - Step 3: Ghost Cursor Setup
    - A. Create Cursor Visual
    - B. Create Cursor Material
    - C. Link to BlueprintInteraction
  - Step 4: Camera Setup
    - Top-Down View (Recommended for Home Design)
    - Isometric View
  - Step 5: Testing
    - Troubleshooting:
  - Step 6: Performance Optimization
    - For Large Grids (50x50+):
    - For Mobile:
  - Step 7: Extending the System
    - Example: Placing Objects
    - Example: Dynamic Grid Resizing
    - Example: Highlight Selected Cell
  - Visual Style Recommendations (Cozy/Furnish Master Style)
    - Grid Lines:
    - Ghost Cursor:
    - Camera:
  - Architecture Benefits
  - Next Steps
  - Support

Blueprint Grid System - Setup Instructions

Overview

This high-performance Blueprint Grid System uses procedural mesh generation for rendering grid lines, avoiding the overhead of LineRenderers or Tilemap. Perfect for simulation and home design games.

Step 1: Scene Setup

A. Create Grid Manager

1. Create an empty GameObject in your scene: `GameObject > Create Empty`
2. Name it `GridManager`
3. Add the `GridManager.cs` component
4. Configure in Inspector:
    - **Width**: 20 (or your desired grid width)
    - **Height**: 20 (or your desired grid height)
    - **Cell Size**: 1.0 (adjust based on your game scale)
    - **Origin Position**: (0, 0, 0) or wherever you want the grid to start

B. Create Grid Visuals

1. Create an empty GameObject: `GameObject > Create Empty`
2. Name it `GridVisuals`
3. Add the `GridVisuals.cs` component (MeshFilter & MeshRenderer auto-added)
4. Configure in Inspector:
    - **Line Width**: 0.02 (adjust for thicker/thinner lines)
    - **Grid Color**: Light gray with alpha ~0.5 for "cozy" look
    - **Generate On Start**: ✓ (checked)
    - **Grid Material**: Assign material (see Step 2)

C. Create Interaction Handler

1. Create an empty GameObject: `GameObject > Create Empty`

2. Name it `BlueprintInteraction`
3. Add the `BlueprintInteraction.cs` component
4. Configure in Inspector:
    - **Raycast Camera**: Drag your Main Camera here
    - **Ghost Cursor**: Create/assign a visual cursor object (see Step 3)
    - **Snap Speed**: 15 (adjust for snappier/smoother feel)
    - **Cursor Height Offset**: 0.05
    - **Hide When Out Of Bounds**: ✓ (checked)

Step 2: Material Setup

Recommended Shader: Unlit/Transparent

For sharp, delicate grid lines with performance:

1. **Create Material**:
    - Right-click in Project: `Create > Material`
    - Name it `GridLineMaterial`
2. **Configure Shader**:
    - **Shader**: `Unlit/Transparent` (for alpha support)
    - **Color**: Light gray (e.g., RGB: 200, 200, 200, Alpha: 128)
    - **Rendering Mode**: Transparent or Fade
3. **Alternative Shaders** (for special effects):
    - **URP**: `Universal Render Pipeline/Unlit`
    - **Custom Shader**: Write a shader with vertex color support if you need per-line coloring
4. **Assign to GridVisuals**:
    - Drag `GridLineMaterial` to the `Grid Material` slot in GridVisuals component

Shader Tips:

- **For glow effect**: Use emission property with low intensity
- **For dashed lines**: Create custom shader with UV-based alpha clipping
- **For depth fighting**: Adjust material's render queue to 3001+

Step 3: Ghost Cursor Setup

A. Create Cursor Visual

1. Create a 3D Object: `GameObject > 3D Object > Plane` or `Quad`
2. Name it `GhostCursor`
3. Scale it down: `Scale: (0.95, 1, 0.95)` (slightly smaller than cell)
4. Position: `(0, 0.05, 0)` (slightly above grid)

B. Create Cursor Material

1. Create material: `Create > Material`
2. Name it `CursorMaterial`
3. Set shader to `Unlit/Transparent`
4. Set color to green with alpha: `RGBA(0, 255, 0, 100)`
5. Assign to the GhostCursor mesh

C. Link to BlueprintInteraction

- Drag `GhostCursor` GameObject to the `Ghost Cursor` slot in BlueprintInteraction component

---

Step 4: Camera Setup

Top-Down View (Recommended for Home Design)

```
Position: (10, 15, 10)
Rotation: (45, -45, 0)
Projection: Orthographic (Size: 10) or Perspective (FOV: 60)
```

Isometric View

```
Position: (0, 20, 0)
Rotation: (90, 0, 0)
Projection: Orthographic (Size: 12)
```

Step 5: Testing

1. **Press Play**
2. You should see:
    - Grid lines rendered as a single mesh
    - Ghost cursor following your mouse
    - Smooth snapping to grid cells
    - Console log when clicking cells

Troubleshooting:

- **No grid visible**: Check if GridVisuals has material assigned
- **Cursor not moving**: Verify Camera is assigned in BlueprintInteraction
- **No clicks detected**: Ensure GridManager is in the scene
- **Lines too thick/thin**: Adjust `Line Width` in GridVisuals

Step 6: Performance Optimization

For Large Grids (50x50+):

1. Use GPU Instancing on grid material

2. Consider Level-of-Detail: hide grid when camera is too far
3. Disable `MeshRenderer` when not needed

For Mobile:

1. Reduce line width to 0.01
2. Use simple unlit shader without transparency
3. Consider generating grid in chunks for very large areas

Step 7: Extending the System

Example: Placing Objects

```
// In BlueprintInteraction.cs, modify OnGridCellClicked:
private void OnGridCellClicked(Vector2Int gridPos)
{
    Vector3 worldPos = GridManager.Instance.GridToWorldCenter(gridPos.x,
gridPos.y);
    Instantiate(furniturePrefab, worldPos, Quaternion.identity);
}
```

Example: Dynamic Grid Resizing

```
// From any script:
GridManager.Instance.Width = 30;
GridManager.Instance.Height = 30;
FindObjectOfType<GridVisuals>().GenerateGridMesh();
```

Example: Highlight Selected Cell

Add to BlueprintInteraction to create a selection highlight system.

Visual Style Recommendations (Cozy/Furnish Master Style)

Grid Lines:

- **Color**: Very light gray/blue (RGB: 220, 230, 240, Alpha: 60-80)
- **Width**: 0.015 - 0.025 units
- **Shader**: Unlit with slight transparency

Ghost Cursor:

- **Valid**: Soft green (RGB: 150, 255, 150, Alpha: 100)
- **Invalid**: Soft red (RGB: 255, 150, 150, Alpha: 100)
- **Shape**: Rounded square or circle (use custom mesh)

Camera:

- **Angle**: 30-45 degrees for isometric feel
- **Distance**: Far enough to see 10-15 cells at once

Architecture Benefits

✅ **Performance**: Single mesh for entire grid (no draw call overhead)
✅ **Scalability**: Works with grids up to 100x100+ cells
✅ **Clean Separation**: Logic (Manager) | Visuals (GridVisuals) | Input (Interaction)
✅ **No Physics**: Mathematical plane raycast (faster than colliders)
✅ **Smooth UX**: Lerped cursor movement for polished feel

Next Steps

1. Create furniture prefabs that snap to grid

2. Implement rotation system (90° increments)
3. Add undo/redo system
4. Create save/load for placed objects
5. Add multi-cell objects (e.g., 2x2 sofas)
6. Implement drag-to-place functionality

Support

If you need help with:

- Custom shaders for grid effects
- Multi-floor grid systems
- Grid-based pathfinding
- Procedural room generation

Ask your Senior Unity Developer! 🎮